Programming Ex. 5: Classification algorithms

Machine Learning

Supervised Algorithms

UNIVERSIDAD DE SAN BUENAVENTURA

Guide #5 – Classification Algorithms

Introduction

The aim of this exercise is to implement a classification algorithm to get a decision boundary that classifies the dataset in two different groups.

To get started with this exercise, there will be included different files that you have to complete to finish this task. Throughout the exercise, you will execute the script *Lab5.py* which calls functions that you will modify; **please do not modify any other file, just follow the instructions of the assignment.**

Files included.

Lab5.py – *main script to execute practice #5.*
[*] reading.py – *Function to read the plain file of data.*
[*] logistic.py – *Function to compute the cost and gradient descent algorithm.*
[*] plotting.py – *Function to plot the different dataset figures.*
[*] sigmoid.py – *Function to compute the sigmoid function for the hypothesis.*
[*] predict.py – Function to compute the model performance and predict a value.
* *Indicates files you will need to complete*.

1. Data reading function

The file *chess_white.csv* contains the dataset required to implement a logistic regression model. The dataset includes multiple columns representing chess tournament results that will be processed to train the model.

To preprocess the data, use np.*loadtxt()* to read the entire CSV file, then extract columns 4 and 5, which correspond to the ELO rating levels of each of the players in the tournament. Instead of using the raw ELO ratings directly, Rebuild your features with the ELO rating level in two more relevant features:

- **ELO difference (x1):** Calculated as ELO_player1 – ELO_player2, representing the skill gap between opponents.
- **ELO average (x2):** Calculated as the average of both players' ELO ratings, indicating the overall strength level of the match.

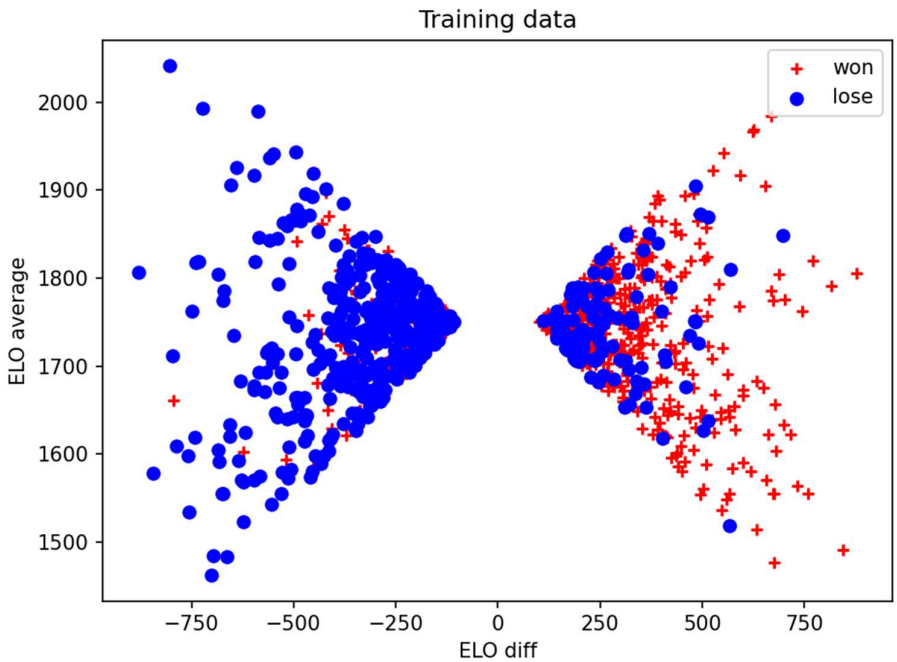
For the target variable, extract column 3, which contains the game results and convert its own binary value: 1 if the result is “1-0” (white player won) or 0 if the result is “0-1” (black player won).

The *data()* function should return two values: x_features (2D matrix containing features x1 and x2) and *Y* (1D array containing the binary target values).

2. Visualizing the dataset

At first, it is a good idea to implement an algorithm to visualize each student's scores on a 2D graph. It is necessary to differentiate with markers players who have won “+” or not “o”.

Once the function ***viewdata( )*** is completed, it will be displayed a figure as follows.



3. Sigmoid Function implementation

To compute the sigmoid function over the dataset, remember that the logistic regression hypothesis is noted as:

$$f_{(\omega,b)} \rightarrow g(z) \rightarrow g(\omega^T x)$$

Where g is the sigmoid function. Complete the function **sigmoid( )**, where for large positives values of x, it returns a value close to 1, while for large negatives values of x, it returns a value close to 0; Finally, for sigmoid(0), the return should be 0,5. Remember that your implementation should work for vectors or matrices too.

To test your function, execute the file *sigmoid.py* that already asks for an entry and compute the corresponding function ***sigmoid( )***.

4. Cost Function and Gradient Descent

Then to implement the cost function in ***costFunction( )***, it should be computed the hypothesis *f(x)* as the following equation.

$$J_{(\omega)} = -\frac{1}{m}\left[\sum_{i=1}^{m} y_{(i)} \cdot \log\left(f_{(x(i))}\right) + (1 - y_{(i)}) \cdot \log\left(1 - f_{(x(i))}\right)\right]$$

After the execution of the cost function with the initial parameters specified in the algorithm, the cost function value should be ≈ 0,693.

Likewise, to implement the gradient descent for a logistic regression algorithm, it should be computed the values of ω with the same structure defined in the linear regression algorithm except for the hypothesis equation used for this model.

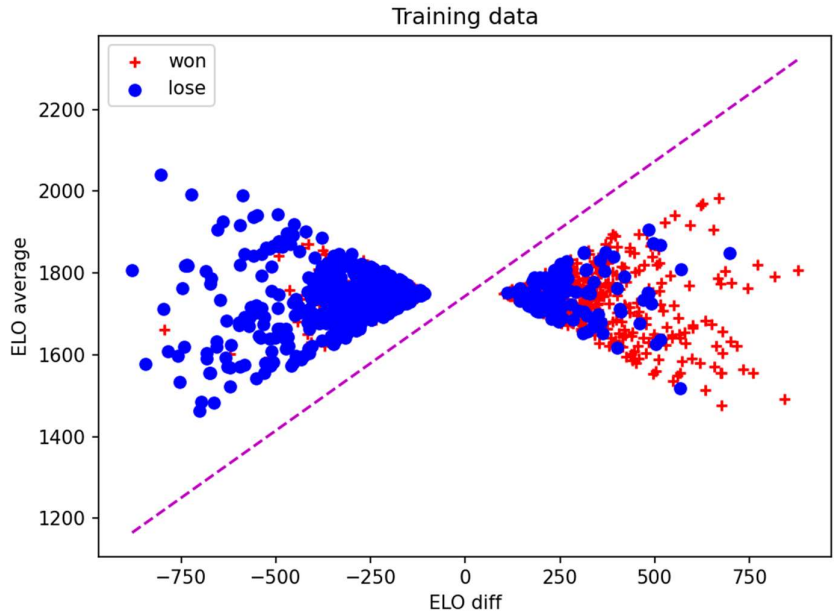
$$\omega_n := \omega_n - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{(x_{(i)})} - y_{(i)}\right) \cdot x_{n(i)}$$

Besides to ensure a correct execution of the gradient descent algorithm, it needs a proper *learning rate value* and its corresponding *number of iterations*. As was done in a previous programming exercise, it can be used a plot to get the change of the loss over each iteration (*it must always decrease)*; do not change the initial parameters of ω.

After completing the ***gradientDescent( )*** function, the cost function value for the optimal value of ω should be ≈ 0.5108.

5. Plotting the decision boundary

With the final values of ω, it should be drawn the decision boundary on the training data for the respective hypothesis; the resulting figure should be similar as shown in the following figure, after completing the function ***viewDecisionBoundary( )***.



To get the decision boundary plot, it should be computed by calculating the hypothesis with the ω values found in the gradient descent algorithm. Note that to draw this decision boundary, the evaluated *z* is equal to zero:

$$f_{(\omega,b)} \rightarrow g(z) \rightarrow g(\omega^T x) \quad \text{where} \quad z = 0$$

$$\text{Therefore} \quad \omega_1 x_1 + \omega_2 x_2 + b = 0$$

6. Model performance

Complete the function ***prediction( )*** to measure the performance of the model, comparing the estimated output of the model with the actual target of the dataset. Therefore, it should be computed a new vector with the computed values of X according to the model found.

Once the estimated output is calculated, the function returns a vector of ŷ stored in the variable *p*, and compare it with the actual target of the dataset *y*, to evaluate how many samples of the dataset are correctly classified according to the known output.

The performance of the model should be ≈ 78%.

Finally, predict the output for a game between two new players:

- White player ELO: 1800
- Black player ELO: 1780